

An Exact-Rational Engine for Kinetic Response Functions: Architecture, Verified Results, and What It Cannot Do

Xavier Callens
Laboratoire SocrateAI, Cagnes-sur-Mer, France

22 September 2026

Abstract

We describe an engine that computes kinetic response functions as exact objects over the rationals $\mathbb{Q}$, without floating-point arithmetic in any derivation step, and we report what it has and has not established. The engine represents a response kernel by its exact rational Taylor coefficients, forms diagonal Padé approximants over $\mathbb{Q}$, and solves the resulting dispersion relations exactly, yielding algebraic numbers with explicit minimal polynomials suitable for export to a proof assistant. Applied to the Landau zero-sound problem in a Fermi liquid, the engine returns the mode velocity as an algebraic number: at $F_0^s = 93/10$, a stand-in for liquid ^3He near saturated vapour pressure, the $[1/1]$ approximant gives the closed form $u = 10/37$, that is $s = \omega/(qv_F) = \sqrt{37/10}$, and the $[4/4]$ approximant agrees with a 60-digit solution of the transcendental dispersion relation to a relative error of 6.8×10^{-10} . We also report a negative result: for weak coupling the method fails outright. The zero-sound mode is then exponentially close to the particle-hole continuum edge, $s - 1 \sim 2e^{-2-2/F_0^s}$, and because the kernel has a logarithmic branch point at $u = 1$, no Padé approximant built at $u = 0$ places a root in $(0, 1)$. The previously advertised capability of tracking the mode “into the Landau damping continuum” by rational Padé is therefore not attainable by this construction. A dedicated section records this and the project’s other retractions, including the fact that a sequence once published here as a nonlinear plasma echo is the Taylor series of $\text{Si}(t)^2/2$.

Contents

1	Introduction	2
2	The Engine	2
2.1	Architecture	2
2.2	How the exactness rule is enforced	3
2.3	Export to Lean 4 and the axiom audit	3
3	Method: exact rational Padé extraction of collective modes	3
3.1	Two kernels, kept apart	3
3.2	Diagonal Padé approximants over $\mathbb{Q}$	4
3.3	The dispersion polynomial and root admissibility	4
4	Results	5
4.1	QV-01: exact zero-sound velocities	5
4.2	Convergence against a 60-digit reference	5
4.3	The weak-coupling threshold law	6
4.4	QVE-02: the nonlinear $\mathcal{O}(\epsilon^2)$ response, and what it is not	6
4.5	Q-RHK-02: symbolic kernel bounds	6
4.6	Q-RIP-03: a definitional identity, reported as such	7

5	Verification and validation	7
6	Limitations, negative results and retractions	8
6.1	A capability the method does not have	8
6.2	Withdrawn claims of earlier versions	8
6.3	Other limitations	8
7	Reproducibility	9
8	Conclusion	9

1 Introduction

Two distinct failure modes threaten machine-assisted theoretical physics. The first is familiar: a language model asked for a derivation may produce fluent, well-formatted mathematics that is simply wrong, and the failure is difficult to detect precisely because the output looks like the real thing. The second is older and quieter: a continuous kinetic problem discretised into `float64` loses, irrecoverably, the analytic structure that determines its qualitative behaviour. Phase mixing, Landau damping and the location of collective modes all depend on branch points and on cancellations that a floating-point pipeline reports as small residual noise.

This project responds to both by refusing floating-point arithmetic in any step that contributes to a derivation. Kernels are carried as exact rational Taylor coefficients; approximants are constructed by exact linear algebra over $\mathbb{Q}$; dispersion relations become polynomials with rational coefficients, whose roots are algebraic numbers with computable minimal polynomials. Floating point appears in exactly one place, clearly quarantined: an external validation script that checks the exact results against high-precision evaluation of the transcendental equations they are supposed to solve.

The contribution of this paper is threefold, and we separate the three deliberately.

1. **The engine** (Section 2): its architecture, and the mechanisms by which the exactness rule is enforced rather than merely documented.
2. **Verified results** (Sections 3–4): exact algebraic zero-sound velocities, a convergence study against a 60-digit reference, and a threshold law reproduced to ten digits.
3. **Limits and retractions** (Section 6): what the method provably cannot reach, and which earlier claims of this project have been withdrawn. This section is not an appendix to the results; it is a result.

We state at the outset what this paper does *not* claim. No experimental data is used anywhere; no comparison with neutron scattering measurements is made. The Lean 4 development verifies arithmetic over $\mathbb{Q}$ and nothing more: it does not formalise kinetic theory, and a Lean theorem in this repository is never evidence for a physical statement.

2 The Engine

2.1 Architecture

The engine is a three-part pipeline. Earlier versions of this repository named its stages after living scientists, which wrongly implied their involvement; the stages are now named for what they do.

`LinearResponseStage` produces exact rational coefficient sequences: kernels of linear response functions, and the linear density perturbation used by the nonlinear protocol.

KineticStage consumes coefficient sequences and performs the kinetic algebra: exact diagonal Padé approximation, exact convolution (Cauchy products) for nonlinear orders, exact root extraction, and symbolic kernel bounds.

AgentSocrate (orchestrator) sequences the protocols, enforces the exactness rule at stage boundaries, and writes results to a persistent store, the *Alexandrie* vault, as JSON containing exact symbolic strings rather than decimal expansions.

2.2 How the exactness rule is enforced

A documented rule that nothing checks is a comment. Three mechanisms make the “no floating point in a derivation” rule operative.

Refusal at the boundary. Entry points that accept a physical parameter reject floating-point input. Supplying F_0^s as a Python `float` raises `ScientificHonestyException` rather than silently rationalising it. This is regression-tested (`test_float_coupling_is_refused`). The rule was added after an earlier revision of this code converted F_0^s to `float` and back via `limit_denominator`, a round trip that silently perturbs the very parameter whose exactness is the point.

Exact representation end to end. Coefficients are `sympy.Rational`; approximant coefficients are obtained by solving the Padé order conditions exactly; roots are returned as exact algebraic numbers, accompanied by their minimal polynomial over $\mathbb{Q}$ when one can be computed. The stored artefacts contain strings such as `13265/11482 - 5*sqrt(4144945)/11482`, not decimals.

Identities tested against closed forms, not against tables. A hard-coded coefficient table is indistinguishable from a fabricated one. The test suite therefore re-derives kernel coefficients from the closed-form generating function symbolically and compares (`test_landau_kernel_matches_closed_form`). A table that was typed in by hand, or produced by a model, fails this test unless it is right.

2.3 Export to Lean 4 and the axiom audit

Exact results are exported to a small Lean 4 development (`lean4_formalization/AgoraPhysics/Protocols.lean`) that checks the arithmetic identities underpinning each protocol. Two points of methodology matter more than the theorems themselves.

First, *grepping for the string `sorry` is not evidence of a complete proof*: a theorem can depend on `sorryAx` through an import. The development therefore ends with `#print axioms` for every theorem, so the build log carries its own evidence, and continuous integration fails if `sorryAx` appears anywhere in that output.

Second, the default build target is the *library*, not the executable. Building the executable forces native compilation of the entire mathlib dependency tree, roughly 5900 additional steps, in order to link a binary that prints a greeting; proof checking does not need it. Correcting the default target is what makes verification in CI practical at all.

3 Method: exact rational Padé extraction of collective modes

3.1 Two kernels, kept apart

Earlier revisions of this project conflated two different functions under the name “Lindhard”. They are now implemented separately and named for what they are, because the conflation produced three mutually inconsistent coefficient tables across the Python code, the Lean development and the prose.

The **Landau zero-sound kernel** is

$$\chi(s) = \frac{s}{2} \ln \frac{s+1}{s-1} - 1, \quad s = \frac{\omega}{qv_F}, \quad (1)$$

whose expansion in the variable

$$u = \frac{1}{s^2} \quad (2)$$

has purely rational coefficients

$$\chi = \sum_{k \geq 1} a_k u^k, \quad \boxed{a_k = \frac{1}{2k+1}} \quad (a_1, \dots, a_6) = \left(\frac{1}{3}, \frac{1}{5}, \frac{1}{7}, \frac{1}{9}, \frac{1}{11}, \frac{1}{13} \right). \quad (3)$$

This identity is verified symbolically in the test suite against Eq. (1). Equation (1) is the kernel that enters the s -wave zero-sound dispersion relation.

A second function appears in earlier tables of this project,

$$g(x) = 1 + \frac{x^2 - 1}{2x} \ln \frac{1+x}{1-x} = \sum_{k \geq 1} \frac{2}{4k^2 - 1} x^{2k}, \quad (4)$$

with coefficients $2/3, 2/15, 2/35, \dots$. It is *not* the zero-sound kernel, it is not used for any dispersion relation here, and we make no claim identifying it with a standard response function of the Fermi liquid. It is retained, and clearly labelled, only because the sequence occurs in previously published tables of this project which readers may wish to reconcile.

3.2 Diagonal Padé approximants over $\mathbb{Q}$

Given coefficients $a_0, \dots, a_{2M}$ of $A(u) = \sum_n a_n u^n$, the diagonal $[M/M]$ approximant P/Q with $Q(0) = 1$ is fixed by

$$A(u) Q(u) - P(u) = \mathcal{O}(u^{2M+1}). \quad (5)$$

The order conditions for $n = M+1, \dots, 2M$ form an $M \times M$ rational linear system for the denominator coefficients $q_1, \dots, q_M$; the numerator then follows from the conditions $n = 0, \dots, M$. Everything is solved exactly; the engine refuses to return an approximant if the system is singular rather than falling back on a least-squares surrogate.

For $M = 1$ the result is small enough to state and to check by hand. From Eq. (3), the single order condition is

$$a_2 + q_1 a_1 = 0 \implies \frac{1}{5} + \frac{q_1}{3} = 0 \implies q_1 = -\frac{3}{5}, \quad (6)$$

so that

$$P(u) = \frac{u}{3}, \quad Q(u) = 1 - \frac{3u}{5}. \quad (7)$$

This order condition is one of the statements verified in Lean (Section 2.3); note that it has content, since it is the equation that determines q_1 rather than a restatement of its definition.

3.3 The dispersion polynomial and root admissibility

The zero-sound mode of the Landau kinetic equation with only F_0^s retained satisfies

$$\chi(s) = \frac{1}{F_0^s}. \quad (8)$$

Replacing χ by P/Q turns Eq. (8) into a polynomial equation with rational coefficients,

$$\boxed{Q(u) - F_0^s P(u) = 0} \quad (9)$$

which the engine solves exactly. An undamped mode requires phase velocity above the Fermi velocity, $s > 1$, that is a real root

$$u \in (0, 1). \quad (10)$$

Roots outside $(0, 1)$ correspond to a phase velocity inside the particle–hole continuum, where the expansion underlying Eq. (1) does not describe an undamped mode; the engine reports their absence explicitly rather than returning the nearest available number. At $M = 1$, Eqs. (7) and (9) give the closed form

$$u = \frac{15}{9 + 5 F_0^s}. \quad (11)$$

4 Results

4.1 QV-01: exact zero-sound velocities

Table 1 lists exact roots returned by the engine. The ${}^3\text{He}$ -like case $F_0^s = 93/10$ is notable for being fully closed-form at $M = 1$: Eq. (11) gives $u = 15/(9 + 93/2) = 10/37$, hence

$$s = \frac{1}{\sqrt{u}} = \sqrt{\frac{37}{10}} = \frac{\sqrt{370}}{10} = 1.923538406167134\dots \quad (12)$$

At higher order the roots are algebraic numbers of degree 2, 3 and 4; the engine returns each with its minimal polynomial over $\mathbb{Q}$, which is the artefact a proof assistant can consume.

Table 1: Exact roots u of the dispersion polynomial (9) admissible in $(0, 1)$. `CRootOf`($p, 0$) denotes the first real root of p ; these are exact algebraic numbers, not numerical approximations.

F_0^s	M	exact admissible root u
93/10	1	10/37
93/10	2	$13265/11482 - 5\sqrt{4144945}/11482$
93/10	3	<code>CRootOf</code> ($102367u^3 - 582876u^2 + 708015u - 150150, 0$)
93/10	4	<code>CRootOf</code> ($2898821u^4 - 30185595u^3 + 78963885u^2 - 66591525u + 12762750, 0$)
30	1	5/53
30	2	$350/337 - \sqrt{101269}/337$
30	3	<code>CRootOf</code> ($6059u^3 - 32697u^2 + 34881u - 3003, 0$)
30	4	<code>CRootOf</code> ($172297u^4 - 1731510u^3 + 4252248u^2 - 3093090u + 255255, 0$)
1	1	none in $(0, 1)$
1	2	$1365/772 - 3\sqrt{44905}/772$

4.2 Convergence against a 60-digit reference

The exact roots are compared with a 60-digit solution of the transcendental equation (8), obtained by bracketed root-finding in `mpmath`. This comparison is the only place floating point enters, and it is external to the derivation. Table 2 reports the relative error in s .

For the strong-coupling cases the convergence is geometric and the agreement at $M = 4$ is 6.8×10^{-10} ($F_0^s = 93/10$) and 7.7×10^{-14} ($F_0^s = 30$). These two cases are asserted by the validation script: a regression that degrades them makes it exit non-zero. The intermediate case $F_0^s = 1$ is reported but not asserted, because its convergence is slow for the reason given in Section 6. Encoding the regime dependence in the test gate, rather than choosing one tolerance loose enough to pass everywhere, keeps the limitation visible.

Table 2: Relative error of the exact Padé root against the 60-digit reference solution of $\chi(s) = 1/F_0^s$. “—” marks entries where the method returns no admissible root; that outcome is discussed in Section 6 and is a property of the construction, not a failure of a particular run.

F_0^s	$M = 1$	$M = 2$	$M = 3$	$M = 4$
1/10	—	—	—	—
1/2	—	—	—	—
1	—	1.49×10^{-2}	3.87×10^{-3}	1.12×10^{-3}
93/10	2.89×10^{-3}	1.79×10^{-5}	1.11×10^{-7}	6.81×10^{-10}
30	3.20×10^{-4}	2.01×10^{-7}	1.25×10^{-10}	7.71×10^{-14}

4.3 The weak-coupling threshold law

Where the Padé construction fails, the threshold behaviour is still available analytically. As $F_0^s \rightarrow 0^+$ the mode approaches the continuum edge according to the standard asymptotic

$$s - 1 \simeq 2e^{-2-2/F_0^s}. \quad (13)$$

Table 3 compares Eq. (13) with the 60-digit root. The ratio is 1.000000 at $F_0^s = 1/20$ and $1/10$, confirming to ten digits both the asymptotic law and, independently, that Eq. (8) is the correct dispersion relation, since an error in the kernel or in the relation would not reproduce an exponential law with the correct constant.

Table 3: Weak-coupling threshold law against the 60-digit reference root.

F_0^s	$s - 1$ (reference)	$2e^{-2-2/F_0^s}$	ratio
1/20	$1.149904453 \times 10^{-18}$	$1.149904453 \times 10^{-18}$	1.0000000
1/10	$5.578936256 \times 10^{-10}$	$5.578936186 \times 10^{-10}$	1.0000000
3/20	$4.383802240 \times 10^{-7}$	$4.383771812 \times 10^{-7}$	1.0000069
3/10	$3.455570071 \times 10^{-4}$	$3.444645119 \times 10^{-4}$	1.0031716

4.4 QVE-02: the nonlinear $\mathcal{O}(\epsilon^2)$ response, and what it is not

The nonlinear protocol takes $\rho^{(1)}(t) = \text{sinc } t$, forms $E^{(1)} = \int \rho^{(1)}$, and evaluates the exact Cauchy product $S^{(2)} = \rho^{(1)}E^{(1)}$ followed by one further integration. The engine returns the exact rational sequence

$$\rho^{(2)}(t) = \frac{1}{2}t^2 - \frac{1}{18}t^4 + \frac{13}{4050}t^6 - \frac{4}{33075}t^8 + \frac{73}{22325625}t^{10} + \mathcal{O}(t^{12}). \quad (14)$$

The arithmetic is exact and the convolution is correct. The sequence is, however, simply the Taylor series of

$$\frac{\text{Si}(t)^2}{2}, \quad (15)$$

which the test suite now asserts term by term. Earlier versions of this project presented Eq. (14) as the discovery of a nonlinear plasma echo. That reading is retracted: an echo is a large-time phenomenon occurring at $t = \tau k_2/(k_2 - k_1)$ and requires two pulses with distinct wavenumbers and a phase space, none of which is present here; a Taylor expansion about $t = 0$ cannot exhibit one. We retain the protocol as what it is, an exact second-order Volterra response, and test it against its closed form so the identification cannot be lost again.

4.5 Q-RHK-02: symbolic kernel bounds

For the phenomenological angular kernel

$$\beta(\cos \theta) = \frac{1}{2} (1 + \cos^2 \theta) \exp\left[-\frac{1}{10} (1 - \cos \theta)\right], \quad (16)$$

the engine computes exactly

$$M_r = 1, \quad m_r = \left[\frac{1}{2} \left(-10 + 3\sqrt{11} \right)^2 + \frac{1}{2} \right] e^{-11/10 + 3\sqrt{11}/10}, \quad (17)$$

$$\Sigma(\beta) = \frac{\pi (910 - 1110 e^{-1/5})}{2} = 1.8988792618620465 \dots, \quad (18)$$

$$\gamma_{\text{bound}} = \frac{m_r}{M_r} + \frac{3}{2} = 1.9512876598772344 \dots \quad (19)$$

Three caveats are essential. First, β is an analytic model chosen for its forward-peaked shape; no measured data enters, and nothing here validates it against ${}^4\text{He}$. Second, the combination in Eq. (19) was previously attributed in this repository to “Theorem 22.6” of Villani [1]; *that attribution has never been checked against the source and we do not assert it*. The method has accordingly been renamed from `apply_theorem_22_6` to `kernel_regularity_bounds`, so that the interface stops asserting a citation the project has not verified. Third, the Lean development carries the rational surrogate $16063/8232 = 1.951287657920311 \dots$, which differs from Eq. (19) by about 2.0×10^{-9} : what is checked there is arithmetic about that rational, not about the algebraic number.

4.6 Q-RIP-03: a definitional identity, reported as such

The remaining protocol evaluates $L_* = 2d$ at $d = 2$ and obtains 4. With L_* defined as $2d$, this establishes that $2 \cdot 2 = 4$ and nothing else. It is not evidence that any Bakry–Émery curvature-dimension constant equals 4, and the earlier claim that it “protects 2D quantum films from Landau damping collapse” has no support in this work. An earlier implementation constructed a `sympy.diffgeom` metric and Ricci tensor and then discarded them, returning $2 \times \text{dim}$; that decorative machinery has been removed and the quantity is labelled a conjecture in both the code and the stored artefacts.

5 Verification and validation

We distinguish three levels of evidence and label every claim in this paper by which level supports it.

Level 1: exact symbolic computation. The kernel coefficients, Padé approximants, dispersion polynomials, roots and minimal polynomials are exact over $\mathbb{Q}$ or $\overline{\mathbb{Q}}$. Ten tests cover these, including the two adversarial ones described in Section 2: the kernel is re-derived from its closed form rather than compared with a table, and float input is required to raise.

Level 2: independent numerical validation. `verification/validate_zero_sound.py` checks the exact roots against 60-digit root-finding and against the asymptotic law (13). It asserts the strong-coupling cases and reports the others, exiting non-zero only if a regime it claims to have solved degrades.

Level 3: proof assistant. The Lean 4 library `AgoraPhysics.Protocols` compiles against `mathlib` at the pinned revision `ec6e08b4`, and every theorem is accompanied by `#print axioms`. The scope of that evidence is narrow and worth stating plainly: it certifies identities such as the $[1/1]$ order condition $a_2 + q_1 a_1 = 0$ and the fact that $u = 10/37$ is a root of $Q - \frac{93}{10}P$. It certifies nothing about ${}^3\text{He}$. Two theorems in the file are labelled vacuous in the source itself: one proves $2 \cdot 2 = 4$ after unfolding a definition, and one has its hypothesis as its conclusion. They are retained, labelled, as a record.

6 Limitations, negative results and retractions

6.1 A capability the method does not have

The central negative result of this paper concerns the weak-coupling regime. The kernel (1) has a logarithmic branch point at $s = 1$, that is at $u = 1$, which is the edge of the particle-hole continuum. A Padé approximant constructed from the expansion about $u = 0$ represents this branch point by a finite set of poles, and for small F_0^s the dispersion polynomial (9) then has no root in $(0, 1)$ at all: at $M = 1$, Eq. (11) gives $u > 1$ as soon as $F_0^s < 6/5$. Higher orders do not repair this, as Table 2 shows for $F_0^s \in \{1/10, 1/2\}$.

The consequence is a retraction of a capability. The protocol registry of this project previously described tracking “the velocity ratio drop $v_{zs} \leq v_F$ into the Landau Damping continuum” by rational Padé approximants. That is not attainable by this construction, and no implementation of it ever existed in the repository. The physically correct statement is that the mode approaches the continuum edge exponentially, Eq. (13), and is exponentially inaccessible to a finite-order rational approximant built at the origin. The engine now reports `root_found: false` with a reason in these cases, and a test asserts that it continues to do so, precisely so that a future change cannot quietly return a spurious root.

6.2 Withdrawn claims of earlier versions

For the record, and because several of these claims are still reachable in the repository’s history and in earlier documents:

1. **The QVE-02 “echo”** is the Taylor series of $\text{Si}(t)^2/2$, not a plasma echo (Section 4.4).
2. **A stored artefact labelled “True 1D1V Vlasov-Poisson phase space simulation”**, with floating-point fields, was never produced by any code in this repository. No such solver exists here.
3. $L_* = 4$ is definitional (Section 4).
4. **The “Theorem 22.6” attribution** is unverified and is no longer asserted by the code.
5. **Three mutually inconsistent coefficient tables** once coexisted for the “Lindhard” kernel: $1/3$, $19/45$ in Lean, $2/3$, $2/15$ in Python, and $1/(2k+1)$ in the prose. The resolution is that $1/(2k+1)$ is correct for the zero-sound kernel (3), the $2/(4k^2-1)$ sequence belongs to the different function Eq. (4), and $19/45$ corresponded to neither and has been removed.
6. **An automated audit of this repository** reported several of these repairs as complete before they were. In particular it stated that the coefficient tables had been unified across Python and Lean while the Lean file was untouched, and that the RPA integration was complete while the code was passing a polynomial into a routine expecting a list of Taylor coefficients, so that the “poles” it produced carried no information. Both are recorded in `RETRACTIONS.md`.

Item 6 is the reason this section exists in this form. An automated pass that writes its own audit report is capable of producing a confident, well-formatted account of work it did not do, which is the first failure mode of Section 1 applied to the act of auditing. The defence is not more prose; it is the executable checks of Section 5, which fail when a claim is false.

6.3 Other limitations

No experimental data is used, so nothing here is validated against measurement. The angular kernel of Q-RHK-02 is a model, not a fit. The Lean development covers arithmetic only. The

strong-coupling value $F_0^s = 93/10$ is a plausible stand-in for liquid ^3He near saturated vapour pressure but is not the result of a fit to data, and the reported agreement is agreement with the transcendental equation, not with an experiment.

7 Reproducibility

All results in Sections 4 and 5 are produced by the following commands from the repository root. Dependencies are pinned in `requirements.txt` (`sympy`, `pytest`, `mpmath`).

```
pip install -r requirements.txt

# exact derivation: writes alexandrie_data/QV-01/zero_sound_results.json
python3 simulations/qv_01_zero_sound.py

# exact symbolic checks (10 tests), including the adversarial ones
python3 -m pytest tests/ -q

# independent 60-digit validation: Tables 2 and 3
python3 verification/validate_zero_sound.py

# proof checking (library target only)
cd lean4_formalization && lake exe cache get && lake build
```

The Lean toolchain is `leanprover/lean4:v4.31.0` with `mathlib` pinned at `ec6e08b4dfb414460f0ee7b4ce967`. Note that `lake exe cache get` is required: compiling `mathlib` from source takes hours, and the build that produced the evidence reported here initially failed because an interrupted clone left the dependency incomplete.

8 Conclusion

An exact-rational pipeline can deliver collective-mode velocities as algebraic numbers with proof-assistant-ready certificates, and at realistic Fermi-liquid coupling it does so accurately: 6.8×10^{-10} relative agreement with the transcendental dispersion relation at $F_0^s = 93/10$, improving geometrically with approximant order. The same analysis shows the method has a hard boundary: the weak-coupling mode sits exponentially close to a branch point and is unreachable by rational approximants built at the origin, which retires a capability this project previously advertised.

The methodological lesson is narrower than the physics but likely more durable. Every claim in this paper that survived did so because something executable would fail if it were false: a kernel re-derived from its closed form rather than compared against a table, a float parameter that raises instead of being silently rationalised, a validation gate that asserts only the regime it can actually solve, and an axiom audit rather than a grep for `sorry`. The claims that did not survive, catalogued in Section 6, were uniformly the ones supported by prose alone, including prose written by an automated audit about its own work.

References

- [1] C. Villani, *Fisher Information in Kinetic Theory*, arXiv:2501.00925 (2025). Cited for the general subject of Fisher-information monotonicity in kinetic theory. The specific attribution of Eq. (19) to a numbered theorem of this work is *not* asserted here; see Section 4.
- [2] H. Godfrin and E. Krotscheck, *The Dynamics of Quantum Fluids*, arXiv:2206.06039 (2022). Cited as a review of ^4He and ^3He dynamics. Neither author has reviewed or endorsed this code.

- [3] D. Pines and P. Nozières, *The Theory of Quantum Liquids*, Vol. I, W. A. Benjamin, New York (1966). Standard treatment of zero sound and of the dispersion relation (8).
- [4] G. A. Baker Jr. and P. Graves-Morris, *Padé Approximants*, 2nd ed., Cambridge University Press (1996).
- [5] The mathlib Community, *The Lean Mathematical Library*, CPP 2020, pinned revision `ec6e08b4`.